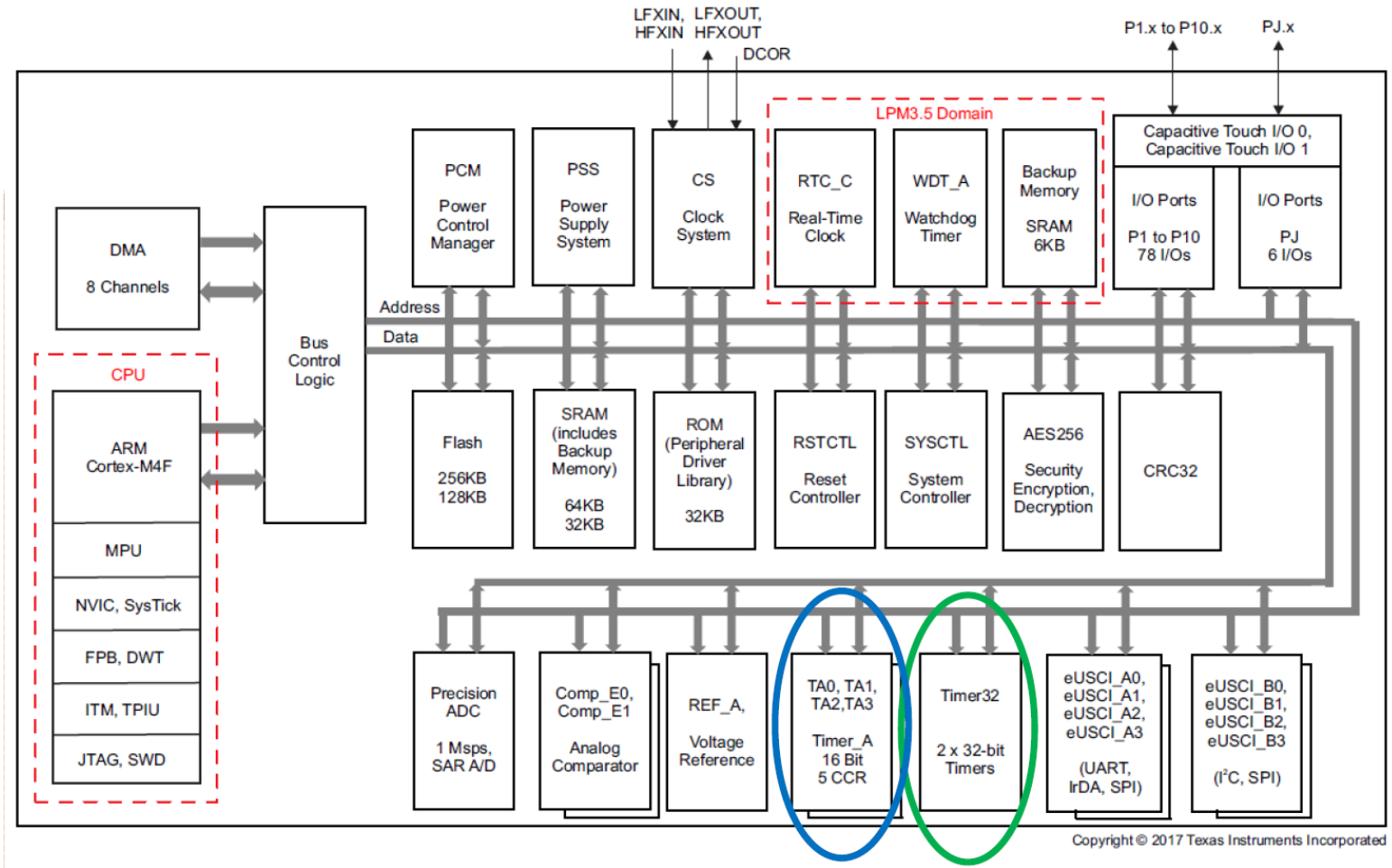
Timer_A Programming

ECE 2564, Embedded Systems

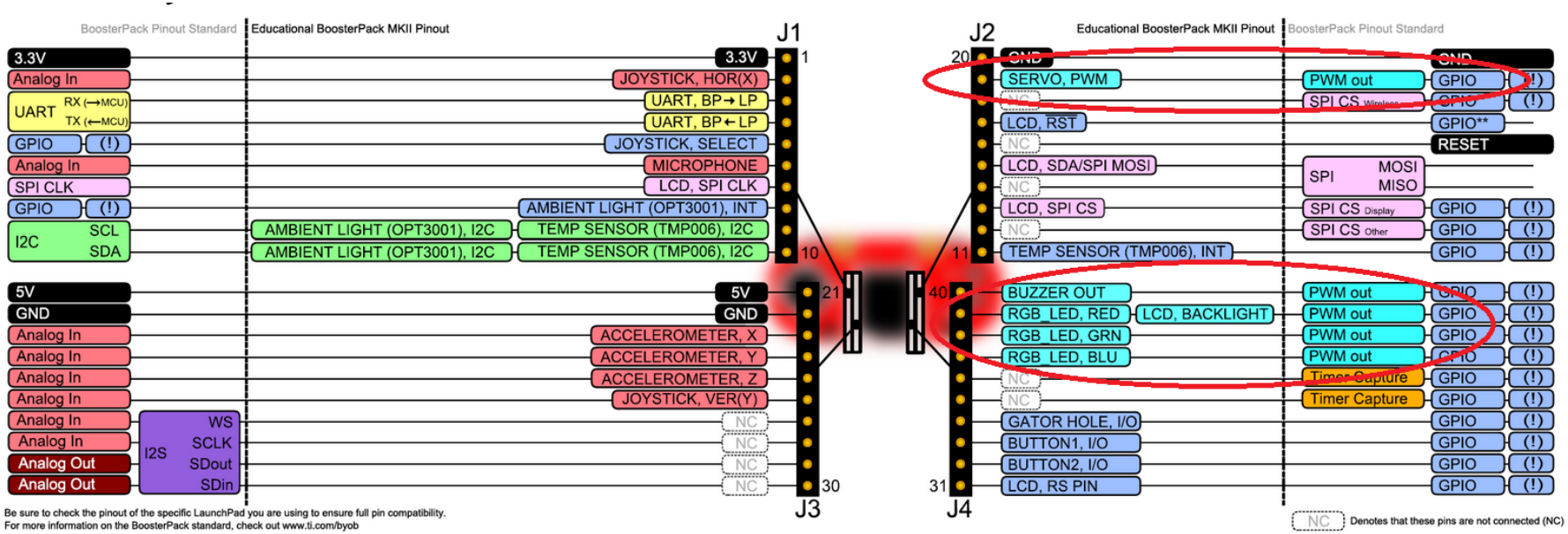
Fall 2025

The MSP432 Functional Block Diagram: Timers



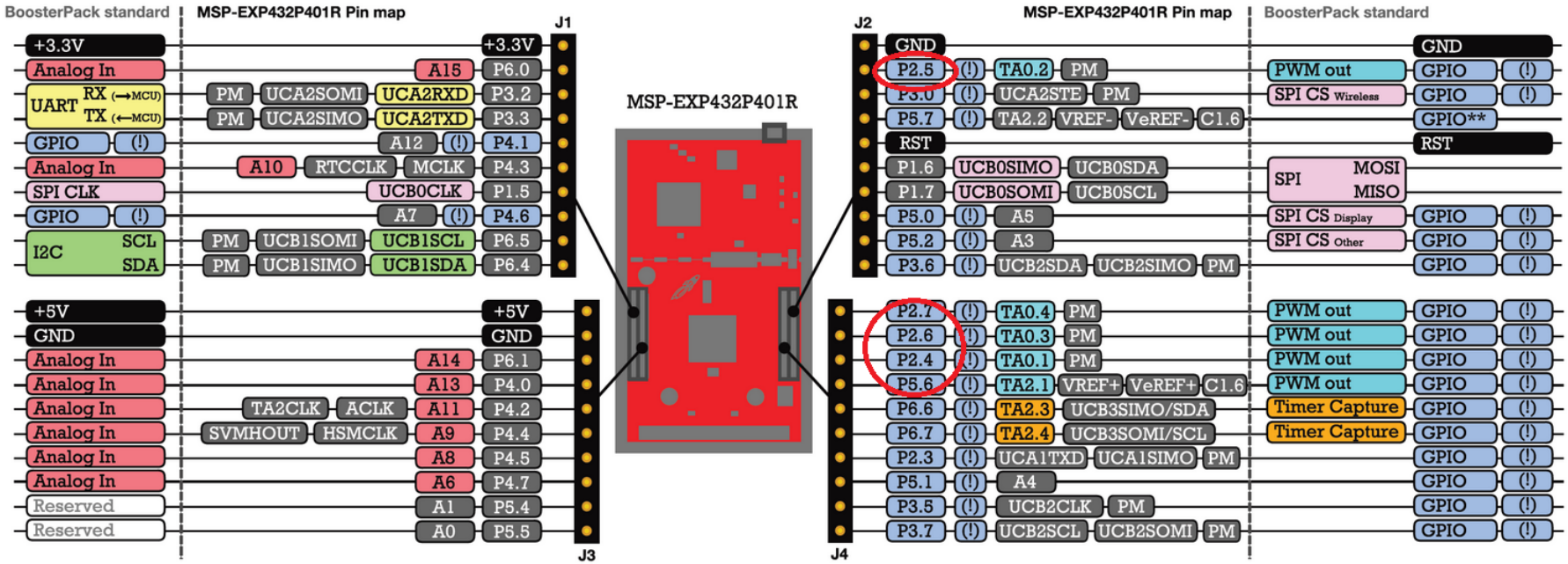
MSP432 Functional Block Diagram: Highlighting the timer modules

Location of Timer_A on BoosterPack



Timer_A on the BoosterPack

Location of Ports and Pins for Timer_A



Ports and Pins for Timer_A on the LaunchPad

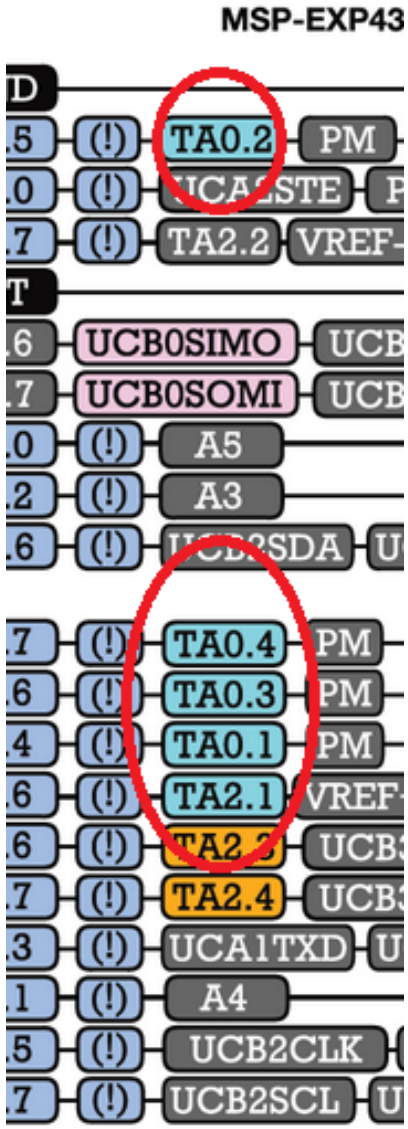
Peripherals Controlled by Timer_A PWM

- Buzzer

- RGB LED, Red
- RGB LED, Green
- RGB LED, Blue

Timer_A Channel Nomenclature

- The peripherals we identified are hardwired to specific port/pin pairs on the MSP432
- These port/pin pairs have a "second" name: `TAi.j`
- `i` is the index of Timer_A (we have four of these)
- `j` is the index of the channel within Timer_A (each Timer_A has multiple channels)



Port/Pin Details[1]

Table 6-75. Port P5 (P5.6 and P5.7) Pin Functions

PIN NAME (P5.x)	x	FUNCTION	CONTROL BITS OR SIGNALS(1)		
			P5DIR.x	P5SEL1.x	P5SEL0.x
P5.6/TA2.1/VREF+/VeREF+/C1.7	6	P5.6 (I/O)	I: 0; O: 1	0	0
		TA2.CCI1A	0	0	1
		TA2.1	1		
		N/A	0	1	0
		DVSS	1		
		VREF+, VeREF+, C1.7(2)(3)	X	1	1
P5.7/TA2.2/VREF-/VeREF-/C1.6	7	P5.7 (I/O)	I: 0; O: 1	0	0
		TA2.CCI2A	0	0	1
		TA2.2	1		
		N/A	0	1	0
		DVSS	1		
		VREF-, VeREF-, C1.6(2)(3)	X	1	1

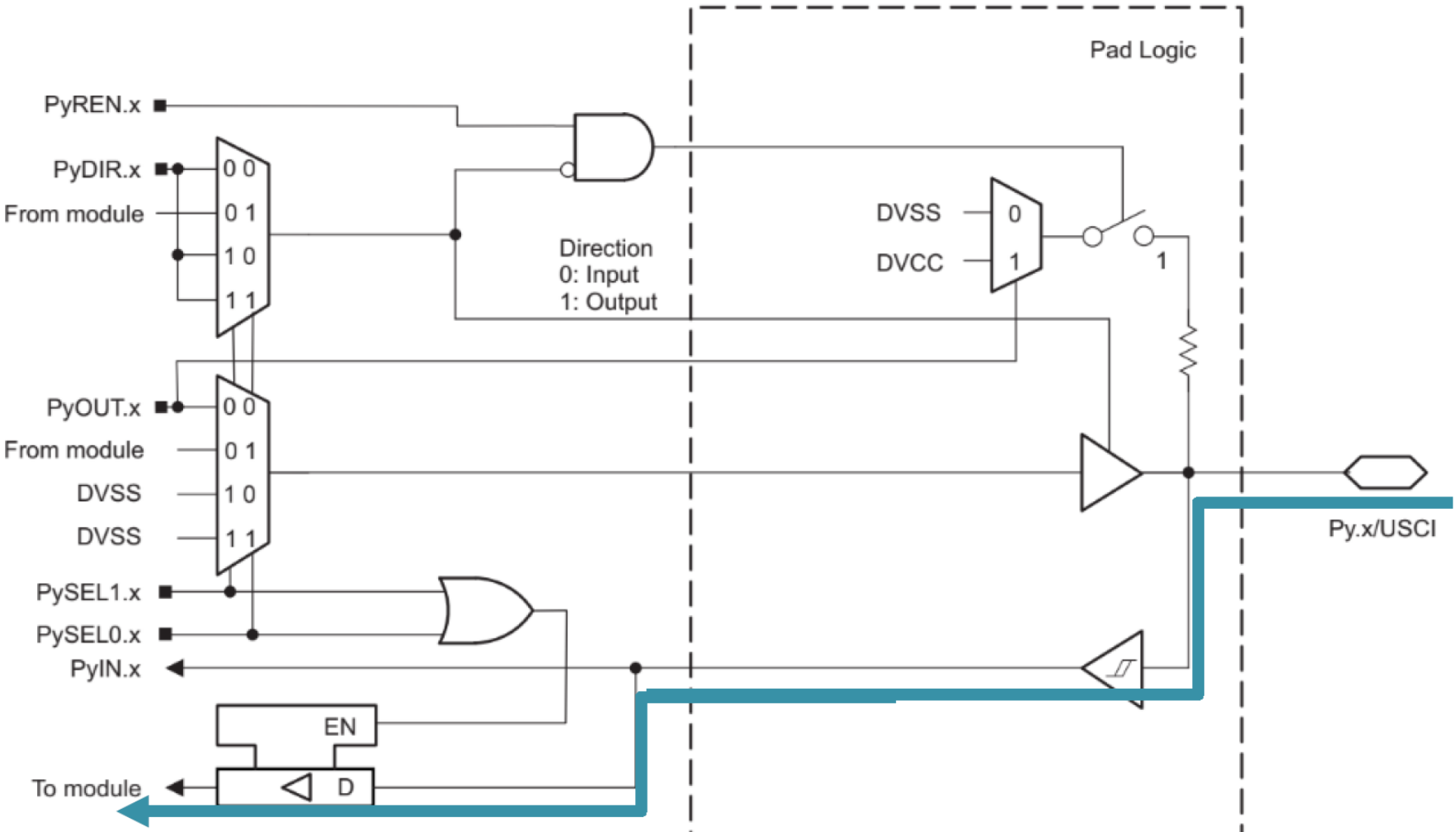
(1) X = don't care
(2) Setting P5SEL1.x and P5SEL0.x disables the output driver and the input Schmitt trigger to prevent parasitic cross currents when applying analog signals.

[1] MSP432 Data Sheet

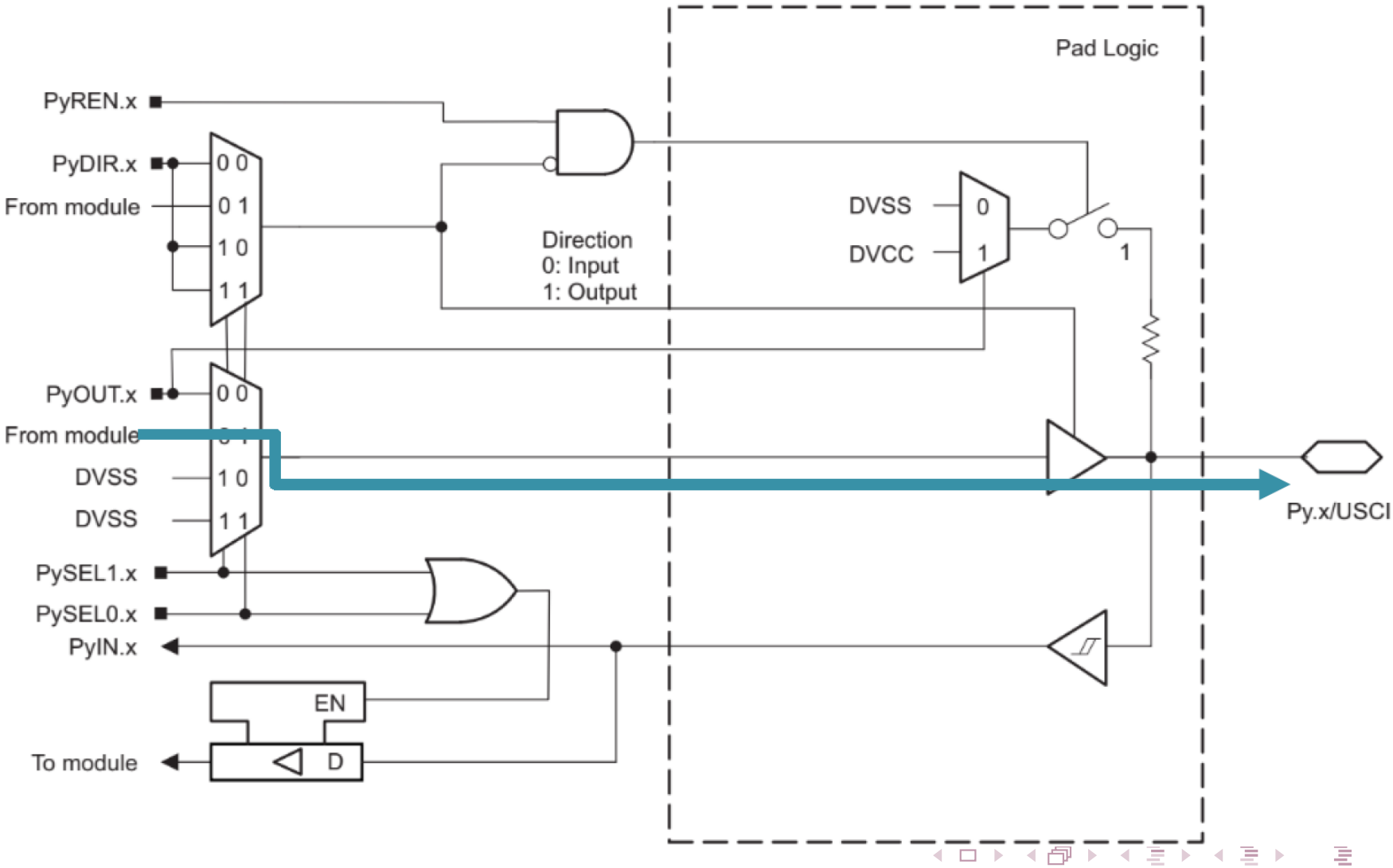
Blue LED on the BoosterPack

- PWM out
- P5.6 (Port:5, Pin6) (TA2.1 – TA2, Channel 1)
  - SEL1 = 0, SEL0 = 1
- This is the *primary* module function

Input Capture



Output Compare



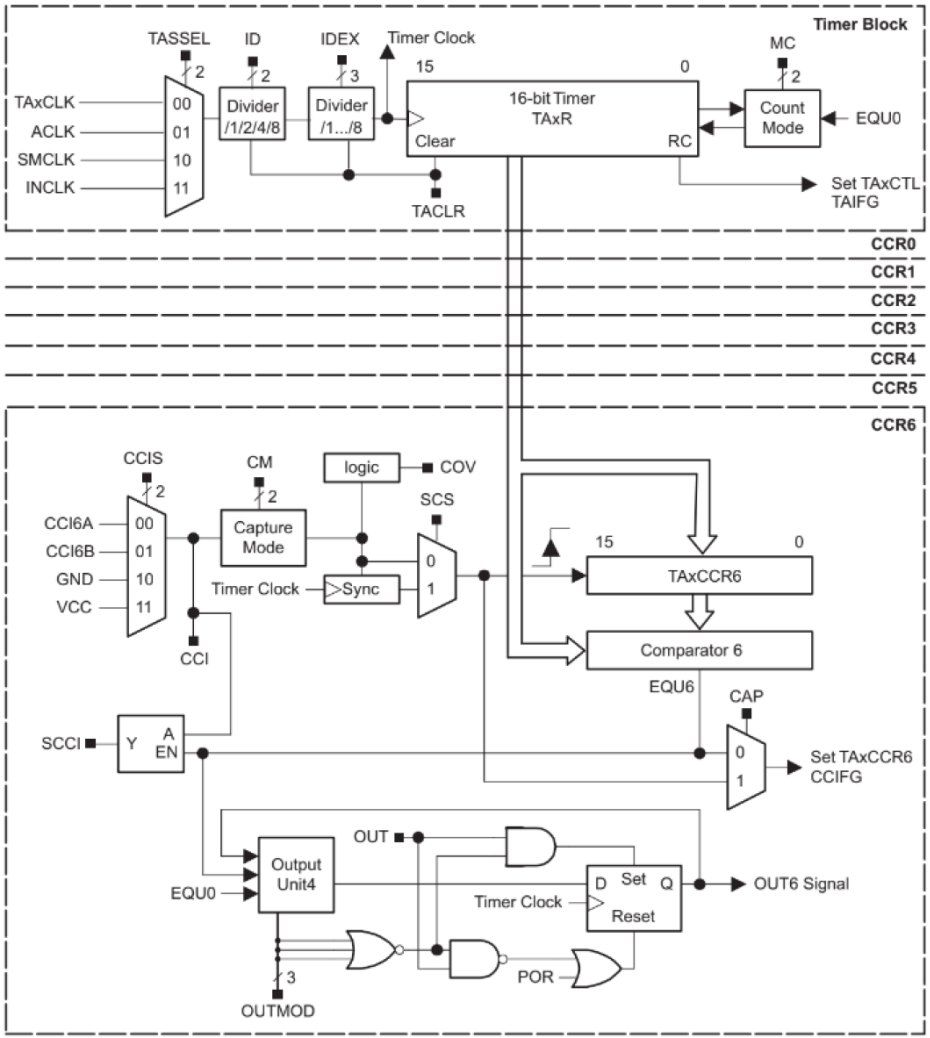
Initialize the PWM

- Connect the GPIO ports
  - Which Timer_A is associated with P5.6? TA2.1

```
void initPWM()
{
    // For the Blue LED, we first configure it as a GPIO
    // and turn it off.  Then we configure it to work as
    // an output channel of a timer.  The blue LED on booster
    // is wired to Port 5, Pin 6, we cannot change this.

    GPIO_setAsOutputPin(GPIO_PORT_P5, GPIO_PIN6);
    GPIO_setOutputLowOnPin(GPIO_PORT_P5, GPIO_PIN6);
    GPIO_setAsPeripheralModuleFunctionOutputPin(GPIO_PORT_P5,
       GPIO_PIN6, GPIO_PRIMARY_MODULE_FUNCTION);
}
```

Timer_A Block Diagram

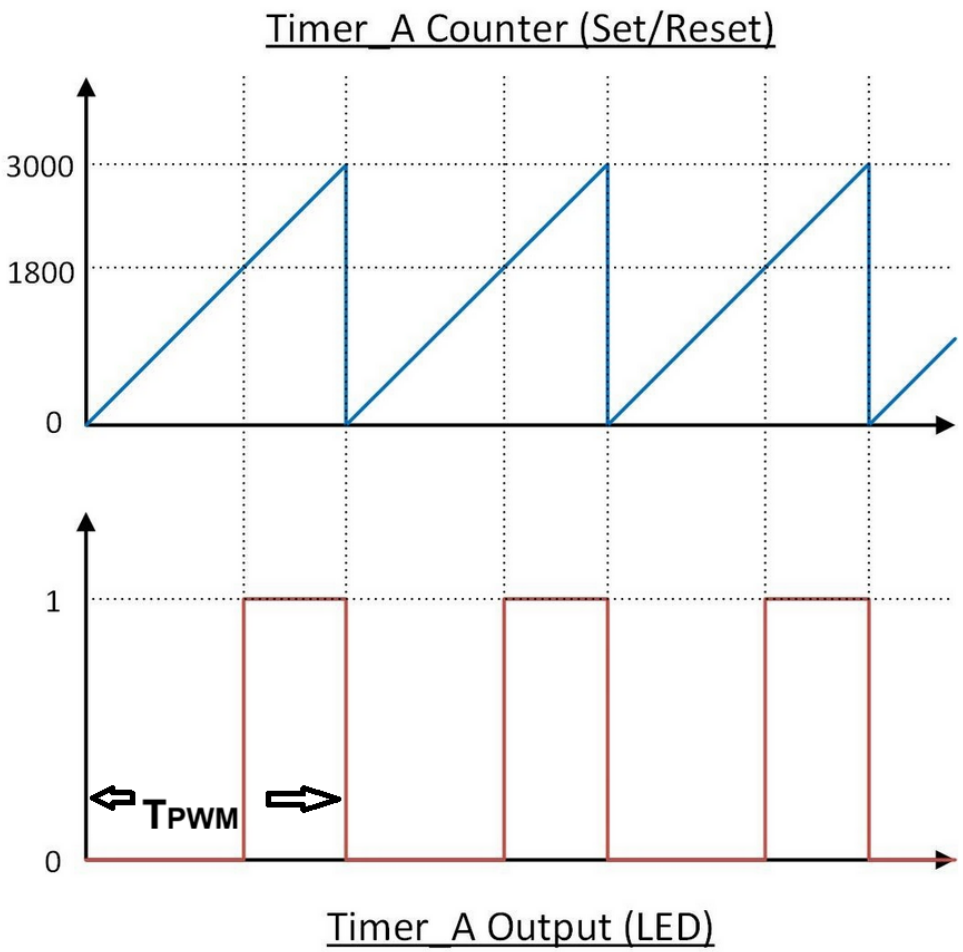


- Clock to timer/counter has a *prescalar* factor
- Six channels sharing the same counter
- Configure the needed registers to set the desired mode

Configuring Timer_A

- Timer_A can be configured in various modes
  - Output compare mode
    - PWM is the most common use
  - Input capture mode
- We should set up a periodic timer whose period matches the period of the PWM pulse
- From the block diagram, Timer_A has a range of options for clock divider, or prescalar, P
- The load value, N, is stored in TAxR, the main counter register

Configuring Timer_A



- CCRn = 1800
- CCR0 = 3000

- What is the period of the PWM?
  - $T_{PWM} = T_{timer} \times 3000$
  - What is T_{timer}?
  - $T_{timer} = T_{sys} \times P$

- Can use five other channels, CCRn, to set other values
  - Achieve different PWM on the same counter

PWM Calculations

For a Timer_A, if the input clock frequency is f_{sys}, the load value is N and the prescalar is P, what is f_{PWM}? NOTE: $f_{timer} = f_{sys}/P$

$$f_{PWM} = \frac{f_{timer}}{N} = \frac{f_{sys}}{N \times P}$$

Example: For $f_{sys} = 3\,\text{MHz}$ and $f_{PWM} = 100\,\text{Hz}$, what should $N \times P$ be?

$$N \times P = \frac{f_{sys}}{f_{PWM}} = \frac{3\,\text{MHz}}{100\,\text{Hz}} = 30000$$

Let $P = 16$ and $N = 1875$

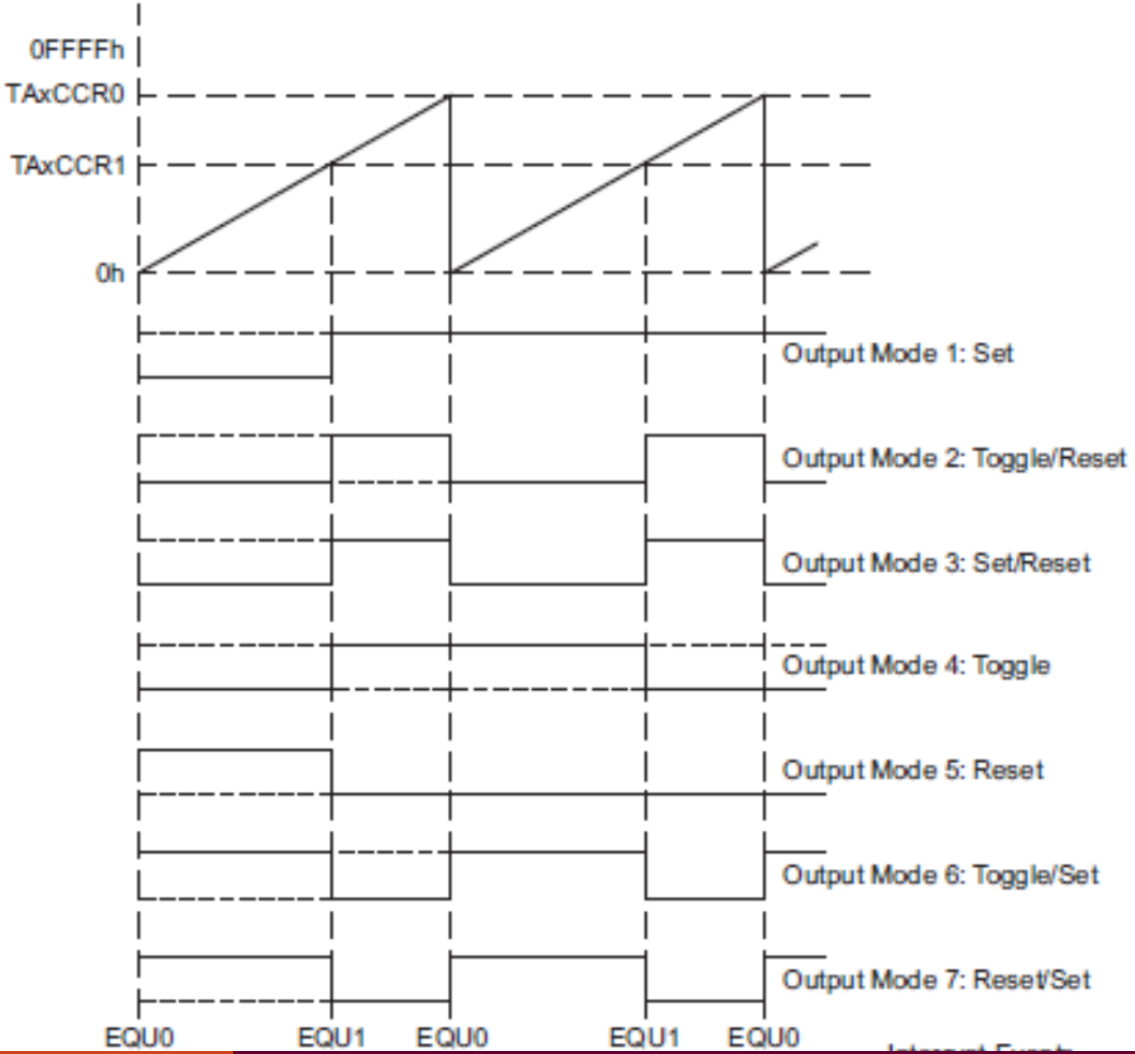
Output Modes

- TAxCCR0 and TAxCCRn registers
  - Where n is the channel, 1 to 5
- Most frequently used: set/reset (outmod = 011) or reset/set (outmod = 111)

OUTMODx	Mode	Description
000	Output	The output signal OUTn is defined by the OUT bit. The OUTn signal updates immediately when OUT is updated.
001	Set	The output is set when the timer *counts* to the TAxCCRn value. It remains set until a reset of the timer, or until another output mode is selected and affects the output.
010	Toggle/Reset	The output is toggled when the timer *counts* to the TAxCCRn value. It is reset when the timer *counts* to the TAxCCR0 value.
011	Set/Reset	The output is set when the timer *counts* to the TAxCCRn value. It is reset when the timer *counts* to the TAxCCR0 value.
100	Toggle	The output is toggled when the timer *counts* to the TAxCCRn value. The output period is double the timer period.
101	Reset	The output is reset when the timer *counts* to the TAxCCRn value. It remains reset until another output mode is selected and affects the output.
110	Toggle/Set	The output is toggled when the timer *counts* to the TAxCCRn value. It is set when the timer *counts* to the TAxCCR0 value.
111	Reset/Set	The output is reset when the timer *counts* to the TAxCCRn value. It is set when the timer *counts* to the TAxCCR0 value.

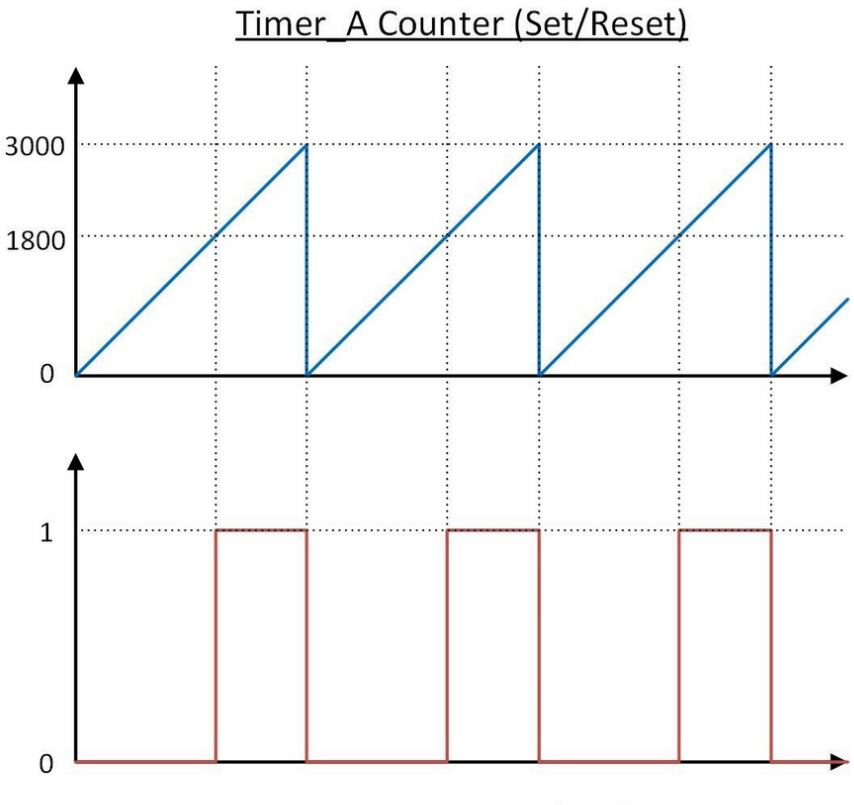
Output Modes Example when Timer in Up Mode

How are CCR0 and CCRn used?



Duty Cycle for Set/Reset Output Mode

$$\text{Duty Cycle: } D = \frac{CCR0 - CCRn}{CCR0}$$

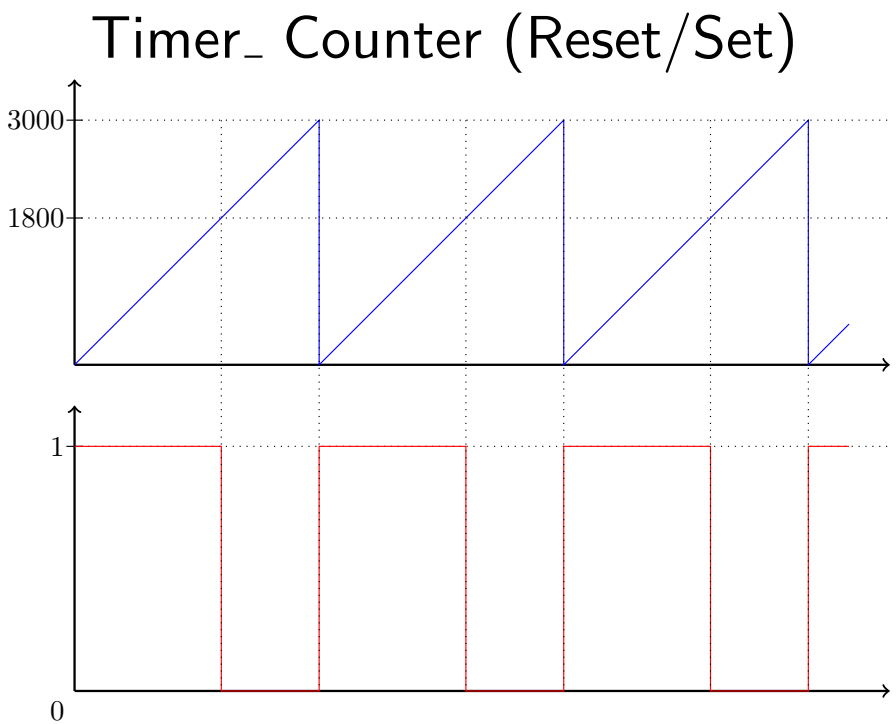


- CCRn = 1800
- CCR0 = 3000

- Set/Reset
  - Set on CCRn
  - Reset on CCR0
  - Duty Cycle: D = 40%

Duty Cycle for Reset/Set Output Mode

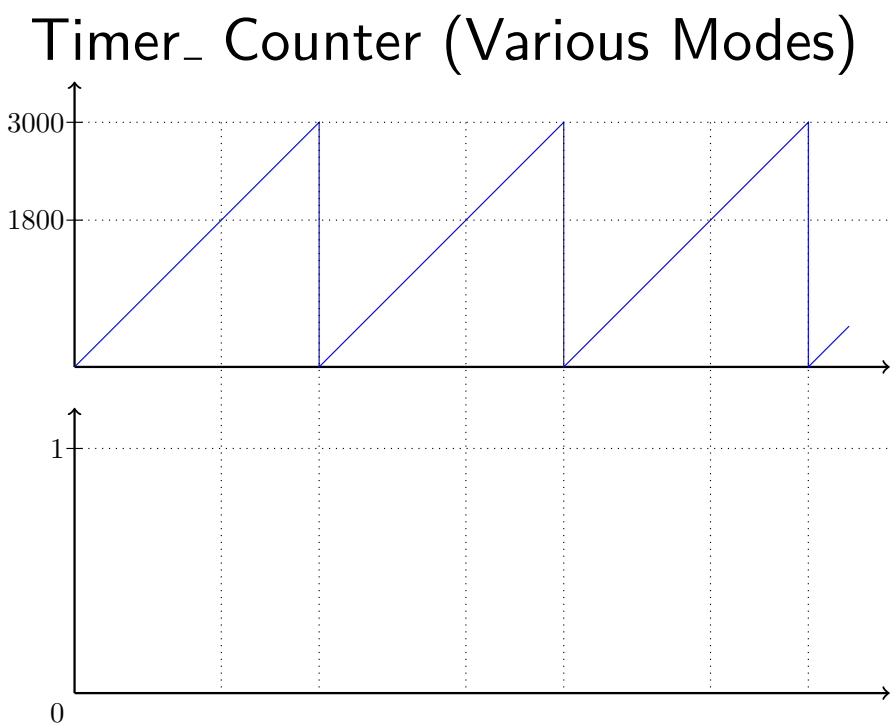
$$\text{Duty Cycle: } D = \frac{CCRn}{CCR0}$$



- CCRn = 1800
- CCR0 = 3000

- Reset/Set
  - Set on CCR0
  - Reset on CCRn
  - Duty Cycle: D = 60%

Behavior of Other Output Modes



- CCRn = 1800
- CCR0 = 3000
- Set: Goes from 0 to 1 exactly once
- Reset: Remains at 0
- Toggle: If the initial value is 0, goes from 0 to 1 exactly once (same as Set)
- Toggle/Reset: If the initial value is 0, it will be the same as Set/Reset

Creating a PWM from f_{PWM}, D and f_{sys}

- Calculate $N \times P = f_{sys}/f_{PWM}$
- Find the most suitable P from the set of possible clock dividers
  - Driverlib name: `clockSourceDivider`
- Compute $N = f_{sys}/(f_{PWM} \times P)$
  - Driverlib name: `timerPeriod`
  - This is also CCR0
- Choose either set/reset or reset/set output mode
  - Driverlib name: `compareOutputMode`
- Using the chosen output mode and D calculate the OC/IC register value
  - Driverlib name: `dutyCycle` (misleading and confusing name)
- Choose the CCR register or channel to use for the PWM
  - This depends on the port/pin you wish to drive

Suitable Choice of P from Possible Clock Dividers[2]

```
#define TIMER_A_CLOCKSOURCE_DIVIDER_1                0x01
#define TIMER_A_CLOCKSOURCE_DIVIDER_2                0x02
#define TIMER_A_CLOCKSOURCE_DIVIDER_4                0x04
#define TIMER_A_CLOCKSOURCE_DIVIDER_8                0x08
#define TIMER_A_CLOCKSOURCE_DIVIDER_3                0x03
#define TIMER_A_CLOCKSOURCE_DIVIDER_5                0x05
#define TIMER_A_CLOCKSOURCE_DIVIDER_6                0x06
#define TIMER_A_CLOCKSOURCE_DIVIDER_7                0x07
#define TIMER_A_CLOCKSOURCE_DIVIDER_10               0x0A
#define TIMER_A_CLOCKSOURCE_DIVIDER_12               0x0C
#define TIMER_A_CLOCKSOURCE_DIVIDER_14               0x0E
#define TIMER_A_CLOCKSOURCE_DIVIDER_16               0x10
#define TIMER_A_CLOCKSOURCE_DIVIDER_20               0x14
#define TIMER_A_CLOCKSOURCE_DIVIDER_24               0x18
#define TIMER_A_CLOCKSOURCE_DIVIDER_28               0x1C
#define TIMER_A_CLOCKSOURCE_DIVIDER_32               0x20
#define TIMER_A_CLOCKSOURCE_DIVIDER_40               0x28
#define TIMER_A_CLOCKSOURCE_DIVIDER_48               0x30
#define TIMER_A_CLOCKSOURCE_DIVIDER_56               0x38
#define TIMER_A_CLOCKSOURCE_DIVIDER_64               0x40
```

[2] From `timer_a.h` in driverlib

Setting Output Mode[3]

```
#define TIMER_A_OUTPUTMODE_OUTBITVALUE          TIMER_A_CCTLN_OUTMOD_0
#define TIMER_A_OUTPUTMODE_SET                  TIMER_A_CCTLN_OUTMOD_1
#define TIMER_A_OUTPUTMODE_TOGGLE_RESET         TIMER_A_CCTLN_OUTMOD_2
#define TIMER_A_OUTPUTMODE_SET_RESET            TIMER_A_CCTLN_OUTMOD_3
#define TIMER_A_OUTPUTMODE_TOGGLE               TIMER_A_CCTLN_OUTMOD_4
#define TIMER_A_OUTPUTMODE_RESET                TIMER_A_CCTLN_OUTMOD_5
#define TIMER_A_OUTPUTMODE_TOGGLE_SET           TIMER_A_CCTLN_OUTMOD_6
#define TIMER_A_OUTPUTMODE_RESET_SET            TIMER_A_CCTLN_OUTMOD_7
```

[3] From `timer_a.h` in driverlib

Setting the Capture/Compare (CCR) Register[4]

```
#define TIMER_A_CAPTURECOMPARE_REGISTER_0    0x02
#define TIMER_A_CAPTURECOMPARE_REGISTER_1    0x04
#define TIMER_A_CAPTURECOMPARE_REGISTER_2    0x06
#define TIMER_A_CAPTURECOMPARE_REGISTER_3    0x08
#define TIMER_A_CAPTURECOMPARE_REGISTER_4    0x0A
#define TIMER_A_CAPTURECOMPARE_REGISTER_5    0x0C
#define TIMER_A_CAPTURECOMPARE_REGISTER_6    0x0E
```

[4] From `timer_a.h` in driverlib

Timer_A_PWMConfig Structure in Driverlib

- All of the Timer_A parameters are collected together in the structure called `Timer_A_PWMConfig` defined in Driverlib
- Remember that `dutyCycle` is CCRn and is confusingly named
- Note: Our MSP432 Booster Pack imposes the use of this macro
  - `#define BLU_CHANNEL TIMER_A_CAPTURECOMPARE_REGISTER_1`

Setting up Timer_A_PWMConfig

```
Timer_A_PWMConfig pwmConfig_blu;

// The source clock is the system clock (3MHz)
pwmConfig_blu.clockSource = TIMER_A_CLOCKSOURCE_SMCLK;

// The clock divider is 64
pwmConfig_blu.clockSourceDivider = TIMER_A_CLOCKSOURCE_DIVIDER_64;

// The # of counter cycles in one PWM round (period of PWM in terms of counter cycle)
pwmConfig_blu.timerPeriod = PWM_PERIOD_CYLCLES;

// The OC mode. We choose this such that we can achieve the waveform we are interested in.
pwmConfig_blu.compareOutputMode = TIMER_A_OUTPUTMODE_SET_RESET;

// The OC value. This is the value that when the counter hits it, something happens to the
// waveform.  Since we chose SET_RESET, at this count the GPIO is set and the LED
// is turned on
// This means the LED was off until hitting this count and so it is the off-cycle count for us
pwmConfig_blu.dutyCycle = BLU_COMPARE_CYCLES;

// The output register used for OC operation.
// The programmer has to choose this such that it feeds to the right pin connected
// to the device needing the PWM.
pwmConfig_blu.compareRegister = BLU_CHANNEL;
```

Bringing Everything Together

- Start the PWM by calling `Timer_A_generatePWM()`
  - The blue LED will be controlled by Timer_A
- The LED will blink according to Timer_A and will not be affected by blocking code
- We can also stop the timer
  - Search Driverlib to find the appropriate function to stop Timer_A
    - `Timer_A_stopTimer(unit32_t timer);`

Bringing Everything Together

```
Timer_A_generatePWM(BLU_TIMER, &pwmConfig_blu);

volatile int i; // Volatile is used here to prevent compiler
        // optimizations from getting rid of this variable.

while (1)
{
    turnOn_LaunchpadLED1();
    for (i = 0; i < (BLOCK_COUNT * BLOCKING); i++);
    turnOff_LaunchpadLED1();

    // No code is needed here for the pulse;  Timer_A
    // takes care of it and the processor is hands off.
}
```

Summary

- Timer_A
  - Basic concepts and modes
  - Programming it
- Pulse Width Modulation
- Duty Cycle